LECTURE 01

01

Introduction to Algorithms

What algorithms are, why they matter, how we represent them, and what makes them valid.

CSC301 • Design and Analysis of Algorithms

Department of Computer Science | Fall 2026

LEARNING OUTCOMES

What should you be able to do by the end?

Lecture 1 follows the CDF outcomes for “Introduction to Algorithms.”

- **DEFINE** an algorithm as a finite computational procedure
- **EXPLAIN** why algorithm study matters for efficient problem solving
- **CLASSIFY** common types of computational problems
- **COMPARE** natural language, pseudocode, flowcharts and programs
- **IDENTIFY** input, output, precision, finiteness and definiteness
- **EVALUATE** correctness and generality using examples
- **REPRESENT** and evaluate an algorithm for summing 1 through n

You already know how to program...

So why study algorithms?

You have 1,000,000 student records.
Find registration number FA23-BCS-115.

FIRST QUESTION?

Not: “Which language?”
But: “What procedure should the computer follow?”



CORE DISTINCTION

Computational problem vs. problem instance

An algorithm solves the general problem—not just one example.

COMPUTATIONAL PROBLEM

Given two integers a and b, return the larger value.

This specifies what must be achieved for every valid input.

PROBLEM INSTANCES

(a, b) = (5, 8) → 8
(a, b) = (17, 4) → 17
(a, b) = (−2, −9) → −2
(a, b) = (6, 6) → 6

PROBLEM = what must be solved → **ALGORITHM = how to solve it**

DEFINITION

What is an algorithm?

An algorithm is a finite sequence of precise, unambiguous steps that transforms valid input into the required output.

FINITE

must terminate

PRECISE

operations are specified

UNAMBIGUOUS

each step has one meaning

INPUT → OUTPUT

transforms data into a result

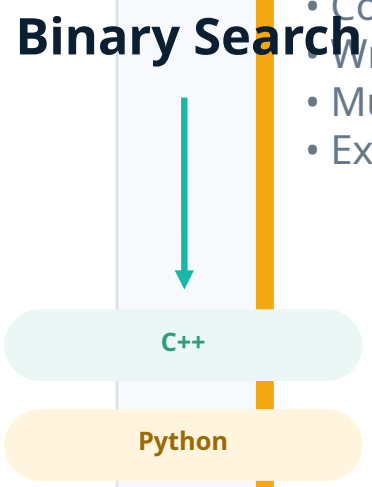
MAX(A): largest ← A[0] → compare remaining elements → update largest when needed → return largest

Algorithm ≠ Program

The algorithm is the solution idea; a program is one implementation of that idea.

ALGORITHM

- Conceptual procedure
- Language independent
- Focuses on logic and steps
- Can be expressed in pseudocode, prose or diagrams



PROGRAM

- Concrete implementation
- Written in a programming language
- Must obey syntax and environment
- Executed by a computer

Same algorithm. Different implementations.

EFFICIENCY MATTERS

Why study algorithms?

Both methods below are correct. But do they use the same amount of work?

METHOD A — REPEATED ADDITION

```
total ← 0
for i ← 1 to n
    total ← total + i
return total
```

WORK GROWS WITH n

METHOD B — DIRECT FORMULA

```
return  n(n + 1) / 2
```

FEW ARITHMETIC STEPS

Correctness is necessary—but algorithm choice can dramatically change resource use.

Algorithms are technology

Better hardware gives us more resources. Better algorithms determine how effectively we use them.



Maps

roads + traffic → route

Search

documents + query → ranking

Scheduling

courses + rooms → timetable

Routing

network + traffic → paths

COMMON PROBLEM CLASSES

What kinds of problems do algorithms solve?

SORTING

Arrange records by GPA

A→Z / low→high

SEARCHING

Find a registration number

locate / retrieve

STRING PROCESSING

Find text in a document

patterns / matching

GRAPH PROBLEMS

Find a route

networks / connections

COMBINATORIAL

Build a feasible schedule

choices / arrangements

GEOMETRIC

Find closest points

spatial relationships

NUMERICAL

Compute GCD or roots

numbers / equations

OPTIMIZATION

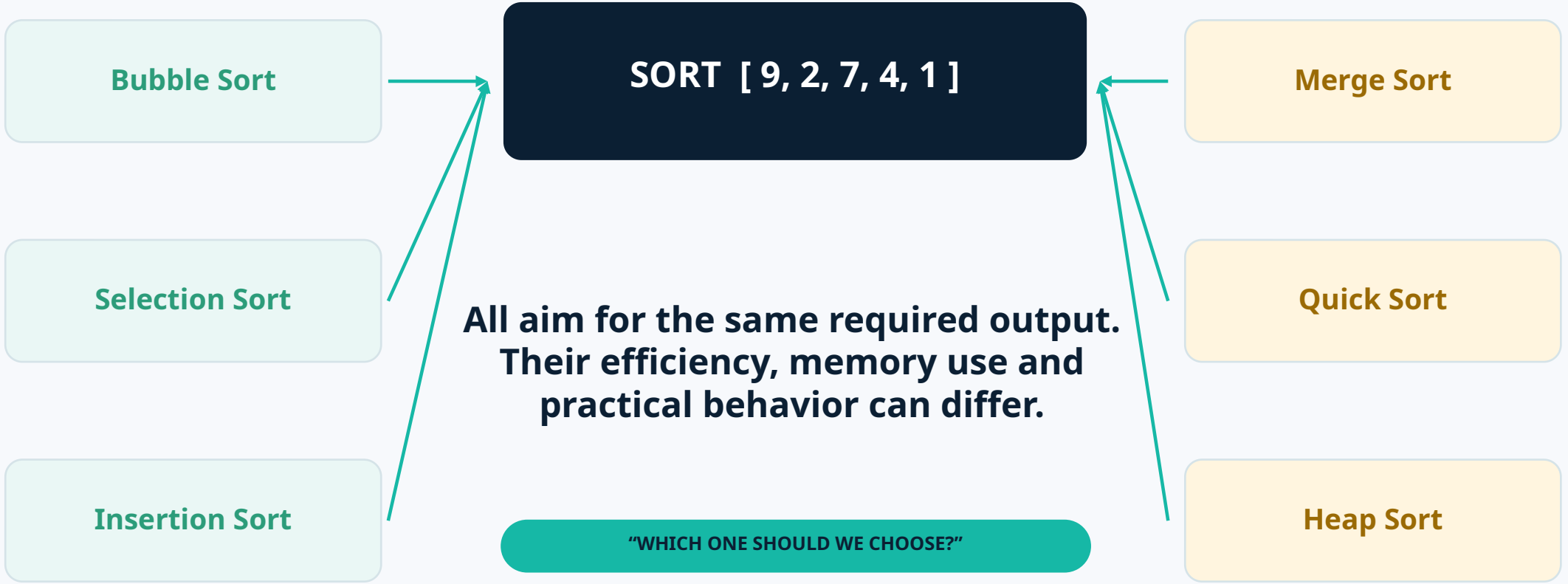
Choose the best feasible solution

cost / value / trade-offs

The problem type often suggests useful data structures and algorithmic techniques.

One problem can have many algorithms

The same sorting problem can be solved in several fundamentally different ways.



That question is central to Design & Analysis of Algorithms.

REPRESENTATION

Four ways to represent an algorithm

1

NATURAL LANGUAGE

Easy for humans

Can become ambiguous

2

PSEUDOCODE

Structured + language independent

Best fit for this course

3

FLOWCHART

Visual control flow

Cumbersome for large algorithms

4

PROGRAM

Executable

Tied to language syntax

In CSC301, pseudocode lets us discuss the algorithm without being distracted by programming-language details.

EXAMPLE: MAXIMUM OF TWO VALUES

Same idea, four representations

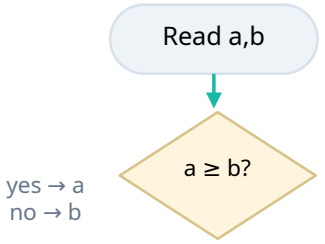
NATURAL LANGUAGE

Read a and b.
If a is at least b, return a.
Otherwise return b.

PSEUDOCODE

```
MAX(a,b)
if a ≥ b
   return a
else
   return b
```

FLOWCHART



PROGRAM

```
int max2(int a,int b) {
  if (a >= b) return a;
  return b;
}
```

Human-friendly ← **increasingly structured and executable** → **Machine-executable**

For algorithm design and analysis, pseudocode gives us the best balance: precise enough to reason about, but independent of a programming language.

REQUIRED PROPERTIES

What makes a procedure an algorithm?

INPUT

Zero or more supplied values

MAX(a,b) receives a and b

OUTPUT

At least one required result

returns the maximum

PRECISION

Operations are accurately specified

total ← total + i

DEFINITENESS

Each step has one interpretation

“sort in nondecreasing order”

FINITENESS

Must terminate after finite steps

loop eventually stops

Bad instruction: “Arrange the values nicely.” → What does “nicely” mean?

EVALUATION

Correctness: does it solve the stated problem?

PSEUDOCODE

```
MAX(a,b)

if a > b
    return a
else
    return b
```

TEST CASES

MAX(10, 4)	→ 10	✓
MAX(3, 12)	→ 12	✓
MAX(7, 7)	→ 7	✓

Testing examples builds confidence—but it is not, by itself, a proof for every valid input.

Formal correctness proofs come next in the course. Today: recognize the idea of correctness.

EVALUATION

Generality: one instance is not the whole problem

Consider the problem: “Given positive integer n, compute 1 + 2 + … + n.”

METHOD A

print 5050

Correct for n = 100.
Not a general algorithm for arbitrary n.

SOLVES ONE INSTANCE

METHOD B

```
SUM(n)
total ← 0
for i ← 1 to n
    total ← total + i
return total
```

SOLVES THE GENERAL PROBLEM

A good algorithm is designed for the complete valid input domain described by the problem specification.

Main example

Sum the integers 1 through n

Problem specification

INPUT	OUTPUT	EXAMPLE
positive integer n	$S = 1 + 2 + \ldots + n$	$n = 5 \rightarrow S = 15$

Now represent the solution as an algorithm—and evaluate its properties.

WORKED EXAMPLE

Algorithm 1: iterative summation

PSEUDOCODE

```
SUM(n)

total ← 0
for i ← 1 to n
    total ← total + i
return total
```

Trace for n = 5

i	total before	total after
1	0	**1**
2	1	**3**
3	3	**6**
4	6	**10**
5	10	**15**

OUTPUT

15

The algorithm terminates after n iterations.

The trace helps us understand behavior for one instance; correctness asks whether it works for every valid n.

ALTERNATIVE SOLUTION

Algorithm 2: use a mathematical identity

$$1 + 2 + 3 + \cdots + n = n(n + 1) / 2$$

ITERATIVE

```
SUM(n)
total ← 0
repeat n additions
return total
```

MORE WORK AS n GROWS

DIRECT

```
SUM(n)
return n(n + 1) / 2
```

FIXED SMALL SET OF STEPS

Both are correct for the stated domain. The difference motivates algorithm analysis later in the course.

CHECKPOINT

Evaluate the SUM algorithm

Input

n

Output

sum 1…n

Precision

specified operations

Finiteness

terminates

Definiteness

unambiguous steps

Correctness

required result

Generality

all valid n

QUICK EXERCISE

Is “keep adding random numbers until the answer looks good” an algorithm? Identify every property it violates.

Discuss for 2 minutes. Be ready to justify your answer.

PRACTICE

Recommended textbook exercises

Use the question numbers below so everyone works from the same edition.

LEVITIN — 3rd edition • Exercises 1.1

Q3(a,b) — Express everyday procedures with the precision required of an algorithm.

Good for: precision, definiteness, natural-language representation.

Optional extension: Q1 — investigate the origin of the word “algorithm.”

CLRS — 3rd edition • §1.1 Exercises

1.1-1 — Give a real-world example requiring sorting or convex hull.

1.1-2 — Identify measures of efficiency other than speed.

1.1-5 — Give one problem needing the best solution and one where an approximate best solution is acceptable.

For submission / discussion: Levitin 1.1-Q3(a,b) + CLRS 1.1-1 and 1.1-2

LECTURE 01 RECAP

Five ideas to carry forward

01 A problem states WHAT must be solved.

02 An algorithm states HOW to solve it.

03 A program is an implementation of an algorithm.

04 Valid algorithms are precise, definite and finite.

05 **Different correct algorithms may use resources very differently.**

NEXT

Fundamentals of Algorithmic Problem Solving

STAIR • specification • constraints • solution choices